Shop AI
the digital waiter

Diners scan a QR code, browse the menu in Thai, Burmese or English, and ask an AI waiter. Staff and the kitchen see calls and orders live. The AI never guesses about allergies.

Team MFU 888 · Aung Nyi Nyi Thant · Chawakron Singkaew · Tharathon Plengsri · Sasiwimon Chatkaew · Natthanon Wongsai

Busy staff miss orders and cannot answer every question

May · front of house

> *“Sometimes several tables call us at the same time… sometimes orders get missed.”*

15–20 customers arrive at once at lunch; three staff do everything.

Chef Kung · chef-owner

> *“A customer tells us not to add fresh pork blood, but after we finish making the dish, we realize that we forgot.”*

A remade bowl: 5–7 minutes, 20–30 baht.

P’Nut · fried-food shop

> *“Customers may say several things at the same time, which makes it difficult for us to remember everything.”*

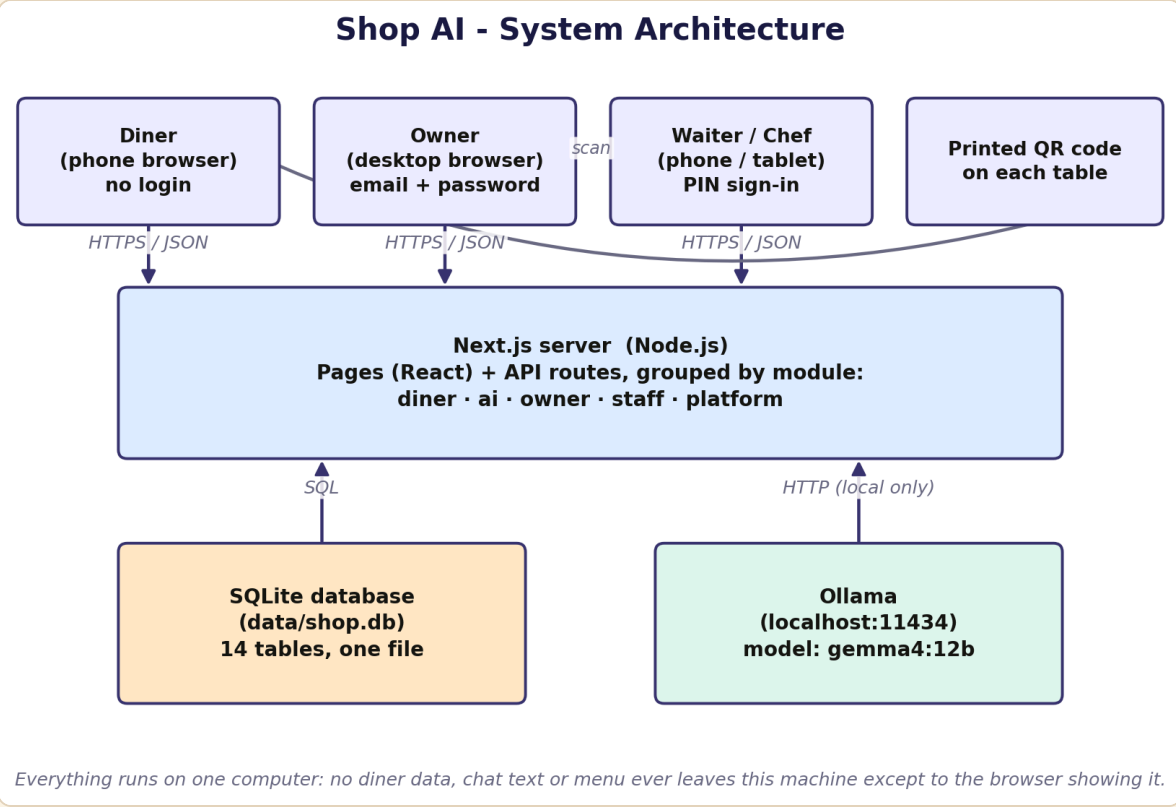
Same questions all evening; a non-spicy dish arrives spicy.

Who has it: front-of-house staff and owner-chefs in small restaurants. **What it costs:** wasted food and kitchen time, slow service, bad reviews.

Source: M1 charter, Gate 1 (three interviews, 22 Sep 2026)

One QR code, three kinds of users, one laptop

- **Diner** (phone, no account): menu in TH / MY / EN, filters, allergy profile, AI chat, picks, call staff, table bill.
- **Staff** (PIN): waiter floor screen and chef board, live every 3 s.
- **Owner** (laptop): menu and 14 allergens, import a menu from a photo, hours, FAQ, AI voice, QR codes, staff, insights.
- Next.js + SQLite + a **local** model (`gemma4:12b` , Ollama). No cloud, no paid API.



Everything runs on one computer: no diner data, chat text or menu ever leaves this machine except to the browser showing it.

Requirements: FR-1 ... FR-10 (M1), SRS 56 Must · 14 Should

3 · KEY IDEA

The AI never decides anything safety-critical

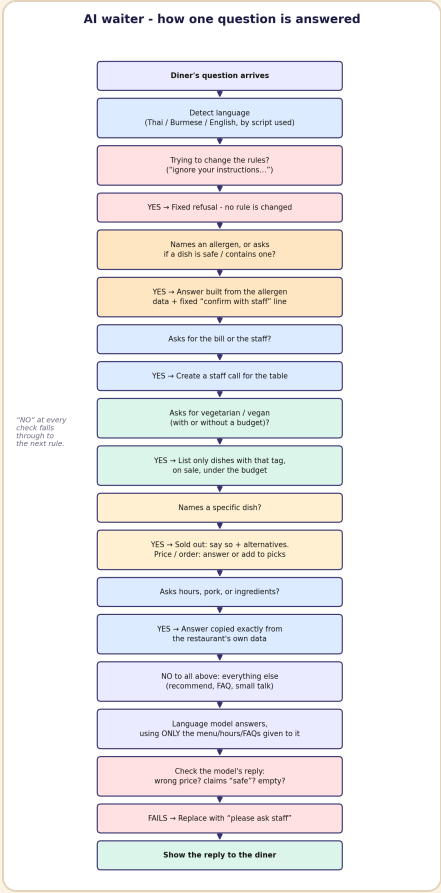
Rules + database (fixed sentences)

allergens · prices · opening hours · sold-out · orders · the bill · vegetarian lists

Language model (open questions only)

“What would you recommend on a hot day?” Then **checked**: no safety claim, no allergen statement, price must match, dish must exist. Otherwise: “please ask the staff”.

Missing allergen data is “not provided”, never “none”. A dish is never called “safe”.



4 · LIVE DEMO (3 MINUTES)

From the QR code to the kitchen

#	DO (PROTOTYPE, NO SERVER)	SHOW: HAPPY, THEN UNHAPPY	REQUIREMENT
1	Diner: switch TH / MY / EN; search “salmon”, then “zzz”	names and prices change; “No dishes found”	FR-1 · DM-1, DM-5
2	Ask AI: “allergic to peanuts”; switch on “AI offline”	no dish suggested (none verified), “confirm with the staff”; fallback with buttons	FR-2 · AI-2, AI-3, AI-11
3	Add a dish, place the order; press Call Staff twice	order number; second call refused	FR-3 · PC-1, PC-4
4	Staff: wrong PIN ×5, then waiter 1111: accept; chef 2222: cook	locked for 60 s; order moves to the kitchen	FR-4, FR-9 · SF-2, SF-3
5	Owner: wrong password; edit a dish; import CSV with a bad row; hours; QR	refused; “skipped 1 invalid row”; closing before opening refused; tables 1–60	FR-5 … FR-8
6	Owner: Insights	the unanswered question is listed	FR-10 · IN-1

Every screen has a **Covers** pill naming its requirement. The real app (`docker compose up`) runs the same story with the local model.

Every feature traces back to the problem and forward to a test

PAIN	FR · SRS IDS	DESIGN	UNHAPPY PATH	TESTS
Same questions; new staff do not know the menu	FR-1 · DM-1 … DM-7, DM-11	UC-1 Browse the menu	Search "zzz": "No dishes found" empty state	ui/dinerx
Same questions all day; allergy risk	FR-2 · AI-1 … AI-13, AI-16 … AI-20	SRS §4.4 chat decision flow	Switch on "AI offline": open question gets a fallback with Browse / Call staff	ai-rules
Tables call at once; orders missed	FR-3 · PC-1 … PC-7	SRS §4.3 order state model, UC-3, UC-4	Empty cart shows "Cart is empty"	orders
Orders missed; special requests forgotten	FR-4 · SF-3 … SF-6, PC-4	UC-8, UC-9, UC-10	Chef and waiter see different tabs (a waiter cannot cook)	ui/staffx
Owner cannot keep the menu right	FR-5 · OA-1 … OA-3, MM-1 … MM-7	UC-5 Manage menu (menu_items, owners)	Wrong password is refused	auth
Menu changes often	FR-6 · MI-1 … MI-4	UC-6 Import a menu (vision model reads, owner reviews)	Rows with a missing name or price are skipped and counted ("skipped N invalid rows")	import
Same questions about hours	FR-7 · ST-1, ST-3, ST-4	settings, FAQs	A closing time that is not after the opening time is rejected ("Closing time must be after opening time")	menu
Diners cannot find the menu	FR-8 · QR-1, QR-2	UC-7 Print QR codes	Going below 1 or above 60 tables is refused ("Tables must be between 1 and 60")	menu
Only staff may take orders	FR-9 · SF-1, SF-2	staff table, SRS SF-2 lockout	A wrong PIN is refused ("Wrong PIN (1 of 5)")	auth
Unseen unmet demand	FR-10 · IN-1 … IN-4	UC-11 Read insights	With no questions yet, the list shows an empty state	owner-insights

Open `prototype/index.html?view=thread` for the full table (happy path, unhappy path, diagram, tests). **108 requirement IDs** are traced to code and tests; a test fails if a row goes stale.

What we measured

1,328
unit and integration tests (74 files); 23 system tests through a real server passed on 29 Oct

29/30
AI questions with the real model on 28 Oct (extended set 30/30, natural questions 34/34; thresholds met)

4.4 s
median for 100 open questions (90th percentile 12.2 s; 99 of 100 within 15 s; target 90%)

0 errors
3 diners + 2 staff screens for 10 minutes; every AI answer within 17.4 s (target 30 s)

< 80 ms
rule-based answers with 30 diners at once (target 1 s)

108
requirement IDs traced to code and tests; CI on Node 22 and 24 and a Docker build

Not measured: a phone over Wi-Fi · real owners (first menu in 30 min) · a restaurant's own computer · photo import on real menu photos.

What does not work yet

- Speed figures are from **one laptop** and a **13-dish menu**; a longer menu means a longer prompt and a slower first answer.
- Three diners chatting nonstop: **about 1 answer in 7** is the fallback message (one model, one at a time).
- The first question after a restart or a menu edit can fall back for up to a minute while the prompt is read.
- No dish has a **restaurant-verified** allergen list yet (145 dishes), so the AI suggests none for an allergy.
- New Thai and Burmese **allergen words need a native speaker**.
- Not tested on real phones, with real owners, or on a public HTTPS server.
- Out of scope by design: online payment, delivery, POS, cloud AI.

What AI helped with, and what people had to decide

Worked

- Tests written first found real bugs: “bill” inside “papaya”, “x4 … x2”, the Thai word for staff read as sesame.
- Measuring beat guessing: the slow first answer was the prompt being read, not the model loading (40–65 s, now warmed at start-up).
- A test keeps the requirements table honest.

People decided

- “AI” vs “staff” wording (corrected twice by a member).
- Burmese replies: four native-speaker reviews.
- No cloud AI, even though it was easier.
- How to merge two lines of AI-assisted work.

[team to complete] what was changed or rejected in the AI's output (Claude Code by one member; Antigravity logged for six tasks on 8 Oct) · docs/AI_USAGE_LOG.md

Who built what

PART	COVERS	MEMBER AND WHERE THE WORK IS IN THE REPOSITORY
1 · Diner web app	FR-1, FR-3 diner side	Tharathon Plengsri (account Tharathon47): [team to complete where in the repository]
2 · AI waiter and backend core	FR-2, backend of FR-5, FR-6, FR-9	Aung Nyi Nyi Thant · `app/src/modules/ai`, `platform`
3 · Owner back office	FR-5, FR-7	Chawakron Singkaew (account chawakron50-lab): [team to complete where in the repository]
4 · Staff and orders	FR-4, FR-9	Natthanon Wongsai (account wujieiei): [team to complete where in the repository]
5 · QR, insights, import screen, tests, docs	FR-6, FR-8, FR-10	Sasiwimon Chatkaew (account noeysasi): [team to complete where in the repository]

Q&A

Questions we expect

Why not let the model answer allergy questions?

It can be wrong and a wrong allergy answer can hurt someone. Allergen sentences come from stored data; the model's reply is checked and replaced if it states allergen facts (AI-17).

Why a local model?

No cost, no API key, no diner data leaves the shop; the charter and the SRS forbid a cloud AI.

What if Ollama is down?

A fallback message at once; menu, picks, staff calls and the rule-based answers still work (NFR-R1).

How do you know the AI is safe?

14 rules each with a test (docs/AI_SAFETY.md), 30-question eval, and a guard on every model reply.

What if a dish has no allergen data?

“Allergen info not provided”, never “none”; it is hidden from diners who set an allergy profile (NFR-S2, NFR-S5).

Is a diner's data stored?

No name, phone or email. The allergy profile stays on the phone; only allergen keys go with the chat and the order (NFR-PR1).

How is access controlled?

Owner login and staff PIN, hashed, signed cookies, every query scoped to the restaurant, login and chat rate limits.

Why SQLite?

One laptop, no server to run; built into Node; write-ahead logging; the file can be copied as a backup.

What would you do next?

Test on real phones and with real owners; native review of every label; a restaurant's own computer; HTTPS deployment.